MATHEMATICAL PROOF-LANGUAGE DESIGN

Heather

Proof-Carrying Mathematical Interfaces

A contract-directed language over Lean

Fourth-round thesis. A mathematical object should bring its proved properties to every legitimate use, without changing its identity. An operation should expose the precise evidence it needs. Automation should connect the two, while preserving the statement, the intended argument, and the scope of every assumption. Heather combines evidence-implicit parameters, snapshot-local elaboration,

relational behavioral contracts, and domain-sensitive finite certificates. The contribution is a concrete integration design and an executed narrow reference model, not another proposal to put unrestricted theorem proving inside type conversion.

Prepared for Vladimir Reshetnikov

6 September 2026

Evidence status: both round-three synthesis reports and selected primary sources were reviewed. Eighteen Python test methods pass. Generated Lean proof candidates are included but were not compiled in this environment. No full repository build, verified frontend, Leant integration, or productivity measurement is claimed.

Abstract

Heather is a proposed mathematical proof language whose central unit is a *proof-carrying interface to an unchanged object*. Rather than asking mathematicians to repeatedly supply already-known side conditions, Heather makes selected proof parameters implicit at use sites and reconstructs their arguments from a bounded, theorem-backed evidence index. Meaning-bearing data remain explicit or uniquely determined before proof search: the chosen ring, topology, measure, region, witness specification, and quantifier structure are not variables that search may adjust to obtain an easier theorem.

This fourth-round proposal accepts the main conclusion of the two round-three syntheses: the first implementation should extend Lean's author-facing typing and elaboration discipline, not its trusted conversion relation. It advances that conclusion in three directions. First, it specifies an evidence-implicit interface with concrete scoping, rewriting, rule-generation, and publication contracts. Second, it develops *realizable affine frames*: domain-sensitive finite certificates that decide affine observation equalities on the actual admissible input space, including empty element types and correlated inputs. A rank argument identifies the number of observations needed for unrestricted affine equality. Third, it supplies an executed Python frontend for a restricted list language, a certificate rechecker, concrete negative witnesses, a small use-site requirement resolver, and ordinary Lean exports.

The article distinguishes mathematical proofs, executed finite tests, historical kernel checks reported by the source documents, and new kernel checks, of which none were obtained here. It works through the ProveIt autonomous-derivative argument, the Fermat repository's finite tensor representability construction, and the inherited exact-quotient benchmark. It also gives an implementation contract for Leant and certified computer algebra, evaluates genuine kernel extension alternatives, and states stopping rules for the separate-language experiment.

Contents

1 The decision: hide routine evidence, not mathematical choices

A mathematician who has introduced a positive real number does not normally write a separate proof of nonzeroness before every division. A mathematician who differentiates an identity on an open set nevertheless relies on a real locality argument: the identity holds near the point, not merely at that point. These two omissions are similar in prose but different in implementation. The first needs a small implication. The second needs the right region, topology, point, and quantified hypothesis before a derivative congruence theorem applies.

Heather aims to make both steps concise *without making either step unaccountable*. Its design principle is:

> Keep the mathematical nouns, choices, and argument landmarks in the source. Infer routine evidence at their interfaces. Retain an ordinary proof of every assertion that the document publishes.

This does not promise that every intuitive mathematical step becomes automatic. It promises that repeated interface work can be delegated under a precise, inspectable contract, and that a blocked operation reports the mathematical premise it still lacks rather than merely exposing a tactic failure.

1.1 What Heather takes from round three

The reports *From Properties to Proof Obligations* and *Nine Refinements* are closely aligned on the foundational choice. Both distinguish properties of a fixed object from choices of structure, use ordinary quantified propositions for behavior, require proof-producing inference, and recommend building an actual use-site interface before adding a large narrative syntax. Both also warn that the proposed interface must beat ordinary Lean supplied with the same new libraries and services [1, 2].

Heather adopts that architecture. It does not rename the four standard refinement rules and call their agreement a new discovery. Nor does it count historical compilation of small contract combinators as implementation of a language. The source reports themselves distinguish these levels of evidence, and their reconciled versions correct important overstatements about finite observations, `List Empty`, theorem counts, and the meaning of compilation [1, 2].

1.2 The proposed increment

The intended fourth-round increment is summarized below. “Executed” refers to this archive’s Python reference model, not to Lean acceptance.

Question	Heather's answer	Present evidence
What should a use site do?	Insert only designated proof arguments under a fixed meaning; expose missing premises and their consumers.	Specified; tiny Nat requirement resolver executed.
What survives rewriting and edits?	Exact proof anchors within one snapshot; checked transport within a context; rebuild after declaration edits by default.	Specified; cross-anchor and stale-epoch refusals executed.
When can finite observations decide behavior?	When a semantic model and a realizable, covering frame determine the demanded relation on the admitted domain.	Written theorems; four list-domain instances executed.
What is the kernel extension?	An author-facing typing discipline first; a primitive refinement former only after a measured encoding bottleneck.	Design and translation argument.
What would justify the language?	Lower authoring or repair cost than equally equipped Lean, without increased semantic-reading errors.	Evaluation design; no human study.

The affine-frame result is a useful mathematical interface theorem, not a claim of historical novelty in linear algebra. Its contribution to this campaign is to turn the empty-type and correlated-observation warnings into a positive, domain-adaptive acceptance method with a precise completeness statement.

2 Source review and a fair Lean baseline

2.1 Snapshots and scope

The repositories were inspected through their GitHub interfaces. The relevant snapshot identities are retained in Appendix A. ProveIt and the Fermat repository resolve to the same commits discussed by the round-three reports. The current Leant `main` returned in this review resolves to `3a40904b`; the syntheses additionally discuss the distinct snapshot `823259f7`. Heather does not silently identify those two versions. Its direct Leant source observations below concern `3a40904b`; claims about the other snapshot remain attributed to the reports [5, 7, 1, 2].

The review is selective rather than a repository audit. It closely studies the two requested synthesis texts and representative implementation files. It does not claim to have rebuilt ProveIt or the Fermat development, proved the whole Fermat theorem here, reviewed every round-three proposal independently, or measured the distribution of verbosity throughout either repository.

2.2 ProveIt: a genuinely local induction argument

The file `AutonomousIteratedDeriv.lean` proves a reusable theorem about an autonomous differential equation. On an open set U, suppose

$$W'(x) = \varphi(W(x)), \qquad G_0(W(x)) = W(x),$$

and the derivative of G_n exists at each relevant $W(x)$ with a supplied value $G_n'(W(x))$. If

$$G_{n+1}(W(x)) = G_n'(W(x))\,\varphi(W(x)),$$

then $D^n W(x) = G_n(W(x))$ for every n and $x \in U$ [3].

The Lean proof performs induction before introducing the point in each branch. In the successor step, it turns the induction hypothesis on U into eventual equality at the chosen point, uses derivative congruence, applies the chain rule, and rewrites by the recurrence. This is not purposeless verbosity: each operation has a mathematical role. The avoidable cost is repeatedly connecting those roles and their parameters by hand. Equally important, the theorem asks for differentiability of G_n only on $W(U)$, not on all of $\mathbb{R}$. A proposed shortening that assumes global differentiability has changed the theorem [3].

The same file already contains a concise corollary obtained by applying this reusable theorem to globally differentiable G_n. Heather should preserve that library-level compression. Once a theorem packages the right mathematics, a one-line application is a good outcome, not an embarrassment for the new language.

2.3 Fermat: structure assembly and a relational guarantee

The file `P2M/Sol/S_Algebra_exists_algHom_equiv_pi.lean` constructs a finite free representing algebra for a finite family of finite free commutative A-algebras. Its proof separates a two-factor tensor universal property from induction over a finite index type. It assembles structure instances, equivalences of homomorphism spaces, and a naturality equation [4].

This example has three distinct costs. There is mathematical construction: use the base algebra for the empty family and a tensor product for a new factor. There is interface plumbing: instantiate module facts and compose equivalences. Finally there is a relational proof: the chosen equivalence commutes with postcomposition. Heather should reduce the second cost and expose the first and third. Naturality is not an adjective attached independently to each component map.

The original theorem explicitly assumes that *every factor* is finite and free as an A-module. The round-three syntheses correctly identify an exposition that lost those hypotheses. Heather's type layer must fail to infer module finiteness when a factor's finiteness is removed; it must not infer it from finiteness of the index type alone [4, 1, 2].

2.4 Leant: association is already treated seriously

Leant's `Synth/Verification.hs` makes `Verified` an opaque, nominally parameterized callback-acceptance receipt. Its documented boundary explicitly distinguishes this receipt from a behavioral theorem, solver certificate, or kernel proof. Candidate variants are tried in order, and only accepted groups consume the success quota [5].

The `PostVerification.hs` module scopes candidate occurrences by a rank-two epoch and validates a proposed permutation against the original receipts. This is useful association machinery: it protects which occurrence was accepted and subsequently selected. It does not, by itself, prove the candidate's mathematics [6].

The Length adapter is similarly explicit. It ties an accepted candidate graph to a sealed query and keeps the product-domain path distinct. Its comments warn that replay of an abstract finite-spine counterexample does not authorize a source-level behavioral refutation or candidate pruning. Heather therefore proposes an additional semantic bridge, not a repair for a bug that the source already disclaims [7].

2.5 Existing facilities are part of the comparison

Lean already has extensible elaboration and proof-producing tactics. Its function-application machinery includes automatic parameters whose associated tactic scripts construct omitted arguments. Thus the novelty of Heather's `requires` interface cannot be merely "a tactic fills a proof parameter" [8, 9].

Mathlib's `fun_prop` already composes registered function-property theorems. Lean's `mvcgen` supports verification-condition generation from program specifications. Library search, simplification, arithmetic automation, and appropriate uses of `grind` are also legitimate controls [10, 11, 12]. The report-reviewed 4.32.0 supply probe has version-specific qualifications, including experimental `mvcgen` support; the current online manual is not the same snapshot as ProveIt's pinned toolchain [1, 2, 8].

Heather's hypothesis is an integration hypothesis: a common interface for requirements, fixed meaning, explanation, scope, and replay may be more useful than independent tactics at independent holes. It must earn that claim by comparison, not by pretending the host lacks automation.

3 What the mathematician would write

3.1 A small vocabulary with different logical effects

Heather reserves different verbs for different commitments. `given` introduces data; `assume` introduces a hypothesis into the stated theorem; `establish` requests a proof; `choose` authorizes construction of a new witness satisfying a fixed specification; `observe` requests a specified view of an existing object. `use` applies a registered mathematical method. The distinctions matter more than these particular English spellings.

For example, the following is *proposed full Heather syntax*, not the implemented four-line grammar of the companion:

```
given x : Real
assume hx : 0 < x
establish reciprocal_cancels : x * (1 / x) = 1
  via field cancellation
```

The author supplies positivity, the target, and the method. The field cancellation interface requests a proof of $x \neq 0$; the evidence engine derives it from `hx`. The generated proof parameter is hidden in the compact view, not absent from the accepted proof. If the assumption is weakened to $0 \leq x$, the nonzero obligation remains open. Heather does not silently add $x \neq 0$ to the theorem's hypotheses.

An assumption introduced during a proof changes its statement unless it is inside an explicit conditional subproof. Consequently an editor command that repairs a failed proof by adding an assumption must present a statement change, not report that it has merely repaired elaboration.

3.2 Keep induction general enough

For the autonomous equation, a compact mathematical argument should resemble:

```
establish derivatives :
  for every n : Nat, for every x in U,
    D[n](W)(x) = G[n](W(x))
proof
  induct n, retaining the quantifier over x in U
  zero:
    conclude by the initial equation
  successor n:
    on U, D[n](W) = G[n] compose W by induction
    differentiate this identity locally on U
    conclude by the chain rule and the recurrence
```

The successor line represents a method application with several proof parameters, not an unrestricted natural-language instruction. Its interface requires equality on a neighborhood of the point. The open-set evidence and membership supply that neighborhood; the derivative hypotheses on the actual range supply the chain-rule inputs. Section 12 gives the full mathematical proof and the obligations represented by these lines.

The compact source must preserve the quantifier over x. A point-specialized induction hypothesis does not suffice for differentiating an identity. Heather therefore makes the retained induction motive part of the elaborated method interface, rather than relying on a language model to guess the missing scope.

3.3 Reason as hint, or reason as obligation

Two annotations have intentionally different meanings:

```
establish P  hint by induction
establish P  via induction on n
```

The first influences search and permits any accepted proof of P. The second requests a proof represented by an induction method node with checked base and step obligations. It is insufficient for the emitted Lean term to contain an unused reference to an induction theorem. Heather validates the designated method's derivation interface, including its conclusion and children.

This mechanism checks an explicit operational interpretation of "by induction", not a human author's private intention. A method registry must say what each mandatory method requires. Some methods may be deliberately broad; others may restrict permissible support or require a particular transformation. The policy is visible in the document.

3.4 Two views, one checked document

The compact view displays theorem statements, mathematical choices, significant intermediate assertions, open conditions, and required methods. The expanded view displays resolved parameters, selected structures, evidence dependencies, search routes, and exported Lean terms. Both views are projections of the same typed document model.

A statement such as "the solution set is complete" cannot be shortened to "a solution exists" in the expanded target, even if the latter is easy to prove. Likewise, a published intermediate assertion must have its own accepted proof even if it is not used by the final theorem. Checking only the final

proof would allow a document to contain unchecked mathematics in its explanatory middle. These publication rules are inherited from the earlier campaign and retained by both round-three syntheses [1, 2].

4 An evidence-aware type discipline

4.1 The judgment and its ordinary Lean meaning

A Heather typing state contains an immutable declaration environment Σ, a dependent local context Γ, a set of selected structure arguments, and an evidence index I. Structure arguments are ordinary data within the elaborated context; listing them separately is an implementation and explanation convenience, not a second logic.

For a fixed Lean term $a : A$, a *facet* is a proposition $P(a)$ together with a retained proof $p : P(a)$. A row Φ is a finite collection of such propositions. It may mention several objects: $\mathrm{Commutes}(f, g)$, $\mathrm{EqOn}(f, g, U)$, or a naturality law for a whole family. The useful source judgment has the form

$$\Sigma; \Gamma; I \vdash t \Rightarrow A \mid \Phi \quad \rightsquigarrow \quad a : A, \quad \pi : \bigwedge_{P \in \Phi} P(a).$$

Here $A \mid \Phi$ is an author-facing classification. Locally, Heather does *not* replace a by a new subtype value every time a fact is learned. Instead, it records ordinary Lean proofs in I. An explicit boundary can package a and selected proofs into a subtype or structure when an API actually expects such a value.

A row is not a collection of informal tags. In particular, `positive` has a fully elaborated proposition containing its ordered structure, zero, and carrier. `integrable` contains its measure, measurable structures, codomain, and function. No matching on the printed word alone is meaningful. The metadata may index a tuple of subjects for efficient retrieval, but the proof's type is authoritative.

4.2 Four elementary rules and two non-elementary obligations

The familiar rules are retained, with their proof terms made explicit:

$$\frac{a : A \quad p : P(a)}{a : A \mid P} \qquad \frac{p : P(a) \quad q : Q(a)}{a : A \mid P \wedge Q}$$

$$\frac{p : P(a) \quad h : \forall x,\, P(x) \to Q(x)}{a : A \mid Q} \qquad \frac{p : P(a) \quad e : a = b}{b : A \mid P}.$$

Their translations use the supplied proof, conjunction introduction, $h(a, p)$, and equality elimination. There is no semantic subtyping search hidden inside Lean's conversion checker. Subsumption is a generated proof argument.

Two less elementary obligations deserve equal prominence. First, changing an *object* requires a relation to the original object or an explicitly specified construction; row conjunction cannot identify two separately chosen witnesses. Second, changing a *context* requires a well-typed substitution for its dependent telescope. A matching printed name, a rewritten string, or a matching cache digest is not such a substitution.

4.3 Property-implicit parameters

Heather introduces an argument role distinct from both ordinary data inference and typeclass search:

```
method cancel_left (x y z : R)
  requires (hx : Regular x)
  requires (hxy : x * y = x * z)
  ensures y = z
```

The declaration denotes an ordinary theorem with explicit proof parameters. At a Heather call site, designated `requires` arguments become goals for the evidence engine. Explicitly supplied proofs always remain available as an escape hatch. The implementation should initially realize this role through Lean metaprogramming and automatic-parameter machinery, plus registry metadata, rather than by modifying the kernel [9].

Only a proposition-valued parameter may have this role. A ring structure, measurable space, enumeration, inverse function, or normalization witness is not a proof argument merely because constructing it is inconvenient. Such data are inferred by the ordinary elaborator when uniquely determined, supplied by the author, or requested through an explicit construction specification.

A method may also have dependent output data. Its semantic shape is then

$$(x : A) \to (h : P(x)) \to \{y : B(x) \mid Q(x, y)\}.$$

The result creates a new y, not a new view of x. Later requirements may mention that exact y and proofs returned with it. An unordered set of guard strings cannot substitute for this dependent binding order.

4.4 Refinement binders are explicit about quantifiers

A source binder

```
given x : A satisfying P(x)
```

introduces $x : A$ and a proof of $P(x)$ as assumptions of the surrounding statement. Accordingly,

$$(\forall x : A \text{ satisfying } P(x),\ Q(x)) \quad \rightsquigarrow \quad \forall x : A,\ P(x) \to Q(x),$$

whereas existential syntax translates to $\exists x : A,\ P(x) \wedge Q(x)$. The word `establish` never has this assumption-introducing effect.

The surface deliberately avoids an ambiguous bare arrow such as "$A \mid P \to B \mid Q$". Two legitimate readings are different:

$$f : A \to B \quad \text{with } \forall x,\ P(x) \to Q(x, f(x)),$$

and

$$f : (x : A) \to P(x) \to B.$$

The former describes a total function with a guarded behavioral guarantee. The latter only provides an evaluation interface when a guard proof is available. Heather requires these two forms to be spelled differently. It never silently extends a refined-domain function to all of A.

4.5 A meaning boundary, not a hash-based proof rule

Before invoking external search, Heather records the elaborated target and its meaning-bearing arguments: carrier types, universe instantiations, chosen structures, fixed objects, quantifiers, relations, regions, measures, precision, and permitted result strength. Proof holes may remain; an explicitly authorized witness hole may also remain under its fixed specification. Other unsolved data holes are resolved or reported as ambiguities before the search is accepted.

This policy prevents a prover from satisfying a request by choosing the zero ring, a different measure, a weaker postcondition, or a convenient branch of an inverse function that the author did not request. It does not prevent ordinary mathematical construction: a request to choose *any* root meeting a stated condition really does authorize a choice among such roots.

The implementation may compute a digest of this record for routing or caching. Acceptance still checks the actual term against the actual target in the actual environment. A digest establishes neither equality of mathematical objects nor a theorem about their behavior. The reference companion follows this separation: the request digest is checked, but certificate data are also reconstructed from the original parsed request.

4.6 Restricted preservation theorem

Theorem 4.1 (Conservative interpretation of the core rules)**.** *Fix a Lean environment and a well-typed interpretation of the source's data, structures, and statement. Suppose a Heather derivation uses only ordinary term formation, the four facet rules above, designated proof-parameter application, and explicit dependent constructions. Suppose every external proof leaf is accepted at its fixed Lean proposition, and every registered rule is applied with well-typed arguments. Then the interpreted derivation produces a Lean term of the interpreted source statement, with no additional axioms beyond those used by its leaves and registered theorems.*

Proof. Induct over the derivation. Ordinary term rules are interpreted by the corresponding Lean constructors. Facet introduction retains a proof; conjunction uses its constructor; consequence applies the registered implication; transport uses equality elimination. A proof-implicit call inserts the recursively constructed proof into an ordinary dependent function application. A construction introduces its actual output and associated proofs into the translated dependent context. Each rule therefore preserves the interpreted target under the translated context. Since no rule introduces an axiom, the axiom dependencies are inherited from the terms used at its leaves and applications. At the root, the result has the fixed interpreted source type. □

The hypothesis about source interpretation is essential. A buggy parser could interpret the wrong quantifier order and still generate a perfectly valid Lean proof of the wrong statement. This theorem does not verify the parser, registry loader, editor, renderer, or Python implementation. It is the mathematical contract those components must eventually implement and test.

5 Behavior is part of an interface, not a confidence label

5.1 Unary and relational contracts

For a fixed total function $f : A \to B$, define

$$\mathsf{Beh}(P, Q, f) \;:\Longleftrightarrow\; \forall x : A,\ P(x) \to Q(x, f(x)).$$

The postcondition is relational: it mentions the input and the actual output. An output-only predicate cannot express preservation of a given list's multiset, a chosen element's orbit, or a computation's relation to its original expression.

Other laws quantify over multiple calls. Monotonicity, Lipschitz continuity, commutation, naturality, and prefix locality are ordinary predicates about the whole function or configuration. Heather permits them as facets without pretending that each can be reduced to a unary postcondition. For example,

$$\mathrm{Lipschitz}(L, f) \iff \forall x, y,\ d(f(x), f(y)) \le L\, d(x, y)$$

retains the two metric structures and the value of L. A proof of this property is an upper-bound certificate, not automatically a proof that L is the least possible constant.

5.2 Consequence with the correct directions

Suppose $\mathsf{Beh}(P, Q, f)$, and suppose

$$\forall x,\ P'(x) \to P(x), \qquad \forall x, y,\ P'(x) \to Q(x, y) \to Q'(x, y).$$

Then $\mathsf{Beh}(P', Q', f)$: apply the first implication to obtain the old input guard, use the original contract, then weaken its postcondition using the second implication. This is the familiar contravariance of preconditions and covariance of guarantees. Calling a contract simply "stronger" obscures which side is being changed.

Heather stores implication theorems rather than an untyped ranking of contracts. Equality, equality on a region, almost-everywhere equality, interval enclosure, and a finite jet are different relations, not confidence scores. Some promotions are possible under additional hypotheses; their exact theorems belong to consumer interfaces.

5.3 Composition retains the intermediate

Let $f : A \to B$ and $g : B \to C$. Assume

$$\mathsf{Beh}(P, Q, f), \qquad \mathsf{Beh}(R, S, g),$$

with bridges

$$\forall x, y,\ P(x) \to Q(x, y) \to R(y),$$
$$\forall x, y, z,\ P(x) \to Q(x, y) \to S(y, z) \to T(x, z).$$

Then $\mathsf{Beh}(P, T, g \circ f)$. Indeed, at an input x satisfying P, retain $y = f(x)$, obtain $Q(x, y)$, establish $R(y)$, obtain $S(y, g(y))$, and use the second bridge. Matching the carrier types $A \to B \to C$ does

not prove either bridge. If g also depends on x or on earlier constructed data, use the corresponding dependent version and preserve that binding order.

A requirement transformer for a fixed f is a sufficient condition generator with a theorem

$$\mathsf{Req}_f(Q)(x) \Longrightarrow Q(f(x)).$$

Composition gives the sufficient transformer $\mathsf{Req}_f(\mathsf{Req}_g(Q))$ for $g \circ f$. Heather does not claim it is a weakest precondition over arbitrary mathematical predicates. Two registered sufficient routes can be incomparable; choosing one need not solve a universal optimization problem over all proofs.

5.4 Sorting and higher-order obligations

A request for $\mathsf{List}(A) \to \mathsf{List}(A)$ admits a constant-empty function. Requiring length preservation still admits operations that discard all input values and repeat a fixed value. Sortedness alone also admits constant empty output. A sorting specification therefore needs a relation to the actual input, such as permutation, together with the required ordering law. Stability, when requested, is a further property rather than an implication of those two clauses.

For a higher-order client, the same discipline applies. A combinator accepting an order-preserving function must receive a proof about the *function passed to it*, under the same orders. A theorem about another extensionally equal function can be used after a checked transport; a convenient function name or a nominal provider tag cannot stand in for that transport.

A polymorphic Lean type does not automatically supply every familiar parametricity theorem. Classical constructions can inspect properties of a type; the Lean reference gives an explicit example. Heather therefore requires an actual uniformity theorem or a restricted syntactic fragment with a proved semantic result before it transfers behavior across element types [13].

5.5 Effects are a separate dimension

A mathematical property of a pure total function, partial correctness of a stateful program, termination, and a search timeout are distinct concepts. For effectful code, Heather should reuse a suitable Hoare-logic interface and Lean's verification-condition infrastructure rather than relabeling every behavioral predicate as an effect [11].

An external solver may time out while a theorem remains true. Conversely, an algorithm may return quickly but fail its postcondition. Heather records operational budgets in the request protocol and logical premises in the proof context. A timeout is never inserted as a mathematical assumption.

6 Bounded inference and the obligation frontier

6.1 A registry is a typed mathematical API

A registered rule records an existing theorem, roles for its parameters, which premises are eligible for property inference, the conclusion pattern, and a selected inference profile. Registration is validated against the actual theorem type. Metadata cannot authorize a conclusion the theorem does not have.

There should be no global instance for every derived fact. Instances select structure or serve existing APIs; the evidence index supplies proof arguments. When an imported theorem genuinely expects

a proposition packaged as an instance, a local, tightly scoped adapter can provide it. This avoids confusing ordinary forward reasoning with open-ended typeclass search.

The index need not be large. Most useful rules should either discharge a frequent missing interface condition or connect facts from two genuinely different packages. A rule that duplicates cheap lookup but increases search branching should be removed unless it improves explanation or robustness.

6.2 How the finite ground fragment is generated

The finite-closure argument of the round-three syntheses starts after a ground rule set has been selected. Heather specifies a conservative first generator rather than treating that step as free [1, 2].

Start from a finite typed pool V containing elaborated subterms of the target, selected local declarations and their types, and explicitly named intermediates. A profile may generate additional terms using a finite operator list, but only up to a declared depth and a total term limit. It may not keep inventing larger terms until something works. Data parameters of a rule are instantiated from compatible entries in this fixed pool; proof parameters become premises. Dependent parameter types are checked in order. Universe and structure arguments must match the fixed elaborated terms.

If the resulting pool has M entries and rule r has a_r eligible data parameters, the raw number of candidate substitutions is bounded by M^{a_r}. Thus $\sum_r M^{a_r}$ is a crude finite upper bound before type filtering. It is not a promise that this generation is cheap. Indexes on types, heads, and roles should reduce the practical enumeration. Each profile also bounds the number of attempts and stops with a distinct generation-exhausted status when its limits are reached.

After constructing the candidate rules, take the backward dependency slice of the demanded goals, including the premises of every retained rule. Forward closure on this fixed slice is predictable. A theorem requiring a witness or intermediate term outside the pool may be missed. That is a limitation of the profile, not evidence against the theorem. The author can introduce the intermediate, choose another profile, supply a proof, or invoke a separately budgeted provider.

6.3 What the closure theorem really establishes

Let $\mathcal{Q}$ be the finite set of propositions in the chosen fragment and let $\mathcal{R}$ contain rules

$$P_1 \wedge \cdots \wedge P_m \longrightarrow Q \quad (P_i, Q \in \mathcal{Q}),$$

each instantiated from a theorem. Starting from accepted evidence S_0, add a conclusion only after all its premises have proofs.

Proposition 6.1 (Finite evidence closure)**.** *A fair worklist procedure adds at most $|\mathcal{Q} \setminus S_0|$ new facts, retains an acyclic derivation for each added fact, and computes the least $\mathcal{R}$-closed superset of S_0. Different fair schedules compute the same set of facts, though they may retain different proofs.*

Proof. Each addition is new and belongs to the finite set $\mathcal{Q}$, proving the bound. Its premises were established earlier, so selecting their existing proofs yields an acyclic derivation and a proof of the conclusion. Every closed superset of S_0 contains each addition by induction over the worklist. Fairness ensures that every enabled rule is eventually considered, so the final set is itself closed. Leastness and schedule independence follow. □

With premise counters, worklist bookkeeping can be linear in the number of facts and premise occurrences. Rule generation, dependent unification, normalization, term construction, proof size, and

kernel checking are excluded from that statement. Heather makes no principal-refinement theorem or global completeness claim for Lean.

6.4 A frontier is a useful partial result

When a demand is not discharged, Heather records the proposition, local context, consuming operation, and a small set of available sufficient routes. For example:

```
Operation: cast the exact quotient into K
Resolved: 4 divides numerator in Int
Open:     the image of 4 is a unit in K
Route:    characteristic zero and field structure suffice
Note:     nonzero in Int is not this destination condition
```

This is not a proof that no other route exists. It is a valid dependency slice for the attempted methods. Finding a globally smallest explanation can itself be expensive; the first implementation should prefer correct and stable explanations over unproved minimality.

An unseeded cycle, such as $P \to Q$ and $Q \to P$, contributes no fact. A conditional result whose guard is G may not be used as unconditional evidence to prove G. A real induction or fixed-point argument must enter through its own theorem-backed method rather than through a cyclic cache entry.

7 Evidence through rewriting, branches, and edits

7.1 The authoritative entry is a typed proof

An implementation entry has the conceptual form

$$(\kappa, \Gamma_0, P, p, \text{origin}, \text{dependencies}), \qquad \Gamma_0 \vdash p : P,$$

where κ identifies an elaboration snapshot. Indices on expression heads, subjects, or normal forms accelerate retrieval, but do not change this invariant. A proof can be offered to a consumer only in a context where its dependencies are valid and its type matches the requested proposition, possibly through an explicit checked adapter.

This design deliberately reuses Lean's contexts and terms. Heather adds an index and explanations; it does not introduce an independent database of mathematical truth whose agreement with Lean is assumed.

7.2 Three cases after simplification

Suppose $p : P(t)$ is available and simplification replaces a displayed expression t by u.

If t and u are definitionally equal in the fixed context, ordinary Lean checking can reuse the proof. If a checked equality $e : t = u$ is available, equality elimination transports p to $P(u)$. If only a weaker relation is known, such as equality almost everywhere, a jet relation, or an order bound, then a consumer-specific theorem must justify the requested consequence.

An equivalence of propositions is also useful: from $h : P(t) \leftrightarrow Q(u)$ and $p : P(t)$, the forward implication yields $Q(u)$. This does not assert $t = u$. The transport trace records which operation

occurred, so the expanded view does not advertise equality of objects when only equivalence of their relevant properties was used.

For dependent indices, the predicate family and its type may need transport together. A term indexed by n cannot be treated as one indexed by m merely because a pretty-printer suppresses the index. The first native prototype should support a few explicit equality and proposition-equivalence adapters, not a general coercion planner.

7.3 Context morphisms and branch export

Let θ be a well-typed substitution interpreting every declaration of Γ_0 in a destination context Δ. It must respect the dependent order: the interpretation of a later declaration is checked after substitution in its type. The ordinary substitution principle gives

$$\Gamma_0 \vdash p : P \quad \Longrightarrow \quad \Delta \vdash p[\theta] : P[\theta].$$

This is the legitimate basis for cross-context reuse. In a native implementation, the destination Lean checker validates the transported term; the context map is not justified by a correspondence between variable names.

A branch adding $h : H$ may use h locally. On leaving the branch, its result P exports as $H \to P$, or under an appropriate case-analysis theorem, not as an unconditional fact for a sibling branch. The same rule applies to an existential witness introduced in a local proof: its scope and dependencies cannot be replaced by a global name with a similar printed form.

7.4 Rebuild first, migrate later

Heather's default after a declaration edit is to discard its local evidence index and reconstruct it from the newly elaborated context. Long-lived caches may retain candidate theorem names, source edits, or proof artifacts for replay, but not a claim that an old local identifier is valid in the rebuilt declaration. A successful replay produces a new acceptance record.

This is intentionally conservative. A persistent proof cache with verified context embeddings may eventually save work, but it introduces a substantial engineering surface. The first prototype should measure rebuild cost before making migration part of the semantics. The reference companion goes further in restriction: it has no equality transport and refuses all stale epochs or cross-anchor reuse. Its tests validate that refusal policy, not a completed implementation of the more general transport design.

7.5 Different failures must remain distinguishable

Heather should distinguish at least four broad outcomes. A demand can have an accepted proof; it can remain unresolved within a profile; a provider or transport can be refused because its schema or context does not match; or the negation of the actual demand can be proved. These outcomes must not share a single undifferentiated failure label.

For behavioral requests, an abstract counterexample is an additional intermediate status. Its promotion to concrete refutation requires the bridge developed below. Budget exhaustion, an unsupported grammar, a corrupted certificate, and an unrealizable abstract assignment are not interchangeable forms of "the program is wrong".

8 Realizable affine frames for behavioral synthesis

8.1 The gap in unrestricted coefficient comparison

The round-three reports develop affine models of list lengths and identify two hazards: a positive abstract length may not be realizable for an empty element type, and independently possible observations may not be jointly realizable under a relational input guard [1, 2].

A useful response is not to abandon finite reasoning. It is to attach the finite reasoning to the *actual input space*. For an admissibility predicate $D : A \to \mathsf{Prop}$, define

$$S = \{x : A \mid D(x)\}, \qquad o : S \to \mathbb{Q}^k.$$

The type S includes the full input configuration and its guard. It may encode multiple correlated inputs, an empty element type, an invariant, or a chosen provider environment. The observation o is interpreted jointly on that type.

8.2 Definition and determining theorem

Definition 8.1 (Realizable affine frame)**.** For a nonempty admissible input type S and observation $o : S \to \mathbb{Q}^k$, a realizable affine frame consists of points $s_0, s_1, \ldots, s_r \in S$ and a coverage theorem

$$\forall s \in S, \quad o(s) - o(s_0) \in \operatorname{span}_{\mathbb{Q}}\{o(s_j) - o(s_0) : 1 \le j \le r\}. \tag{1}$$

The frame need not be linearly independent. Its points are actual admissible inputs, not merely satisfying assignments in an over-approximate abstract model.

Theorem 8.2 (Domain-sensitive affine determination)**.** *Let $(s_0, \ldots, s_r)$ be a realizable affine frame for $o : S \to \mathbb{Q}^k$. For any affine maps $F, G : \mathbb{Q}^k \to \mathbb{Q}$, the following are equivalent:*

$$\forall s \in S,\ F(o(s)) = G(o(s)), \tag{2}$$

$$\forall j \in \{0, \ldots, r\},\ F(o(s_j)) = G(o(s_j)). \tag{3}$$

The same assertion holds for affine maps to a vector space over $\mathbb{Q}$.

Proof. The forward direction follows because each s_j belongs to S. For the reverse direction, set $H = F - G$ and write $H(v) = c + L(v)$, where L is linear. Equation (3) implies $H(o(s_0)) = 0$ and

$$L(o(s_j) - o(s_0)) = H(o(s_j)) - H(o(s_0)) = 0.$$

For any s, coverage expresses $o(s) - o(s_0)$ as a linear combination of these differences. Linearity gives $L(o(s) - o(s_0)) = 0$. Therefore

$$H(o(s)) = H(o(s_0)) + L(o(s) - o(s_0)) = 0,$$

as required. The argument only uses addition and rational scalar multiplication in the codomain, so it also applies to vector-valued affine maps. □

The two directions explain exactly where realizability enters. Coverage of the source image is sufficient for positive transfer from a spanning family of abstract points. Requiring that the family itself is realized additionally makes failure at one of those points a source-level counterexample, and yields the equivalence above. A certificate should not conflate the two requirements.

8.3 Lifting from a model to the exact program

Let e be a candidate in a specified expression grammar, with concrete semantics $\mathsf{eval}(e, s)$ and observation ℓ of its result. Suppose a proved model theorem supplies an affine map F_e satisfying

$$\forall s \in S, \quad \ell(\mathsf{eval}(e, s)) = F_e(o(s)). \tag{4}$$

Suppose a requested target observation is represented by G. Then Theorem 8.2 reduces the universal contract

$$\forall s \in S, \quad \ell(\mathsf{eval}(e, s)) = G(o(s))$$

to the frame equalities. A failed frame equality gives the concrete admissible input s_j, not merely an abstract tuple.

To accept a rendered Lean candidate f, one more theorem is needed:

$$\forall s \in S, \quad f(s) = \mathsf{eval}(e, s),$$

or an adequate direct equality of the requested observations. The expression that was modeled must be the expression the user actually selected. A proof about a convenient surrogate, or a provider's claimed length law, does not close this last bridge.

8.4 Four exact list-domain instances

For lists, observe their lengths as nonnegative integers embedded in $\mathbb{Q}$. The following frames have elementary coverage and realization proofs.

Admissible inputs	Observed frame	Why it covers and is realized
k independent lists over an inhabited type	$0, e_1, \ldots, e_k$	Every length tuple is a sum of basis directions; empty lists and singleton lists realize the points using a supplied element.
k lists over an empty type	0 only	Every list is empty, so the image is a singleton. No positive length is admissible.
Two inhabited-type lists with equal lengths	$(0,0), (1,1)$	Every observation is $n(1,1)$; paired empty and paired singleton lists realize the frame.
k inhabited-type lists of even lengths	$0, 2e_1, \ldots, 2e_k$	Every admitted tuple is a nonnegative integer combination of these directions; length-two lists realize them.

For example, the length models of

$$(x, y) \longmapsto x +\!\!+ y \quad \text{and} \quad (x, y) \longmapsto y +\!\!+ y$$

are $n_1 + n_2$ and $2n_2$. They are unequal on all of $\mathbb{N}^2$ but equal on the diagonal domain $n_1 = n_2$. Testing $(1, 0)$ as a negative input in that domain would discard the admissibility guard. The frame instead uses $(1, 1)$, a pair of actual lists satisfying the guard.

Likewise, constant-empty output and the identity have models 0 and n. On $\mathsf{List}(\mathsf{Empty})$ their source behavior is identical and the one-point frame proves the observation law. On $\mathsf{List}(\mathbb{N})$ the singleton input is a concrete negative witness. This is a change in the domain of the specification, not a change in the soundness of arithmetic.

An empty *admissible input type* S is a separate case from an empty *element type*. When S is empty, every universal contract is vacuous, proved from an emptiness theorem, and there is no base point s_0. By contrast, List(Empty) contains the empty list and has a valid one-point frame. A native interface should have distinct constructors for these cases.

8.5 Why affine length models are valid for the chosen grammar

Consider the list-expression grammar

$$e ::= x_i \mid [] \mid [a] \mid e_1 +\!\!+ e_2 \mid \mathrm{reverse}(e),$$

where the singleton constructor is available only when its element a is supplied. Assign the models

$$x_i \mapsto n_i, \quad [] \mapsto 0, \quad [a] \mapsto 1, \quad e_1 +\!\!+ e_2 \mapsto F_{e_1} + F_{e_2}, \quad \mathrm{reverse}(e) \mapsto F_e.$$

Lemma 8.3 (Length interpretation)**.** *Every expression in this grammar has an affine length model* $F_e(n) = c_e + \sum_i a_{e,i} n_i$ *with natural coefficients, and* $|\mathsf{eval}(e, \rho)| = F_e(|\rho_1|, \ldots, |\rho_k|)$ *for every well-typed input environment* ρ*.*

Proof. Induct on e. An input has its observed length. Empty and singleton lists have lengths zero and one. The length of an append is the sum of lengths, so adding the induction hypotheses adds constants and coefficients. Reversal preserves length, so it preserves the induction hypothesis unchanged. Each constructor preserves the asserted affine form and its interpretation. □

Differences of two natural-coefficient models are interpreted over $\mathbb{Q}$ when applying Theorem 8.2. Equality of their rationally embedded lengths is equivalent to equality of the original natural lengths. This scalar change is an explicit model bridge, not an implicit reinterpretation of the source program's arithmetic.

8.6 The number of observations is controlled by rank

Proposition 8.4 (Size and minimality of a determining family)**.** *Let* S *be nonempty and let the affine hull of* $o(S) \subseteq \mathbb{Q}^k$ *have dimension* r*. There exists a realizable affine frame of* $r + 1$ *points. No family with fewer than* $r + 1$ *observed points determines the restrictions of* all *affine maps* $\mathbb{Q}^k \to \mathbb{Q}$ *to* $o(S)$ *by equality of their values.*

Proof. Choose $s_0 \in S$. The vector space spanned by the differences $o(s) - o(s_0)$ has dimension r. A basis can be selected from this spanning set, giving r further realized points and the required coverage.

Now take $m < r + 1$ proposed observed points. If $m = 0$, two different constant maps already show failure. Otherwise their affine hull has dimension at most $m - 1 < r$. It is a proper affine subspace of the affine hull of $o(S)$. Choose an affine functional that vanishes on the smaller hull but not on the larger one: translate by one selected point, extend a basis of the smaller difference space, and take a coordinate functional on an added basis vector. Extend it to $\mathbb{Q}^k$. This functional and zero agree on all selected points but disagree at some point of $o(S)$, proving the lower bound. □

This is an existence and information-content statement, not an algorithm for deciding arbitrary admissibility predicates or discovering their affine hulls. The proof can use mathematical choices that a computational interface must supply separately. Nor is the lower bound necessarily sharp for a

smaller candidate grammar: it concerns all affine maps, not only the maps a particular synthesizer happens to produce.

Nevertheless, the proposition suggests a useful engineering principle. Input guards can lower the dimension of the observation space and hence reduce both the certificate size and the number of relevant negative cases. Heather should model that reduced space rather than independently abstracting each input and then trying to repair impossible counterexamples afterward.

8.7 What the theorem does not authorize

Affine determination decides *equalities*. Agreement with an inequality at a base point and basis points is not a universal nonnegativity proof: the affine function $1 - n$ is nonnegative at $n = 0, 1$ but negative at $n = 2$. A positivity service needs a different certificate, for example cone-based reasoning with its own coverage and sign conditions.

Similarly, length equality proves neither permutation nor sortedness. A function that branches on an unmodeled property of its input is outside the chosen grammar. Finite observations of that unrestricted function cannot be promoted by borrowing the affine theorem for another grammar. A future extension can use other finite-dimensional feature models, but each requires its own exact denotation and coverage proofs.

9 Leant integration: from candidates to retained behavior

9.1 A request contract

Heather should submit a construction request only after the intended type and behavioral specification have been fixed. A request includes the elaborated candidate type, the proposition about the candidate, selected structures, local assumptions, admissible input domain, permitted providers, resource limits, and the original source location. For a length request, it additionally identifies the observation grammar and any domain-frame package to be used.

The result must distinguish a candidate that has merely passed a callback from a candidate with retained behavioral evidence. Leant already makes the former boundary explicit in its `Verified` abstraction [5]. Heather's stronger acceptance object is conceptually

$$(f, p), \qquad f : T, \quad p : \mathsf{Spec}(f),$$

plus the environmental and source-association information needed to replay those terms. A host-language receipt may refer to the Lean declaration or proof artifact, but the receipt's constructor is not a replacement for that proof.

The nominal epochs and exact-occurrence handling in Leant's post-verification stage are useful foundations for routing this record. They should be retained, not flattened to a string-to-Boolean map. Association checks and proof checks remain separate obligations [6].

9.2 A concrete affine handoff

For the restricted list fragment, the proposed pipeline is as follows. Parse and elaborate the selected candidate once. Produce a typed grammar expression e and a correspondence theorem for that exact candidate. Derive its length model using the grammar's interpretation theorem. Select a proved frame

for the actual admissible input type. Check the finite affine equalities or reconstruct an admissible source counterexample. Finally, retain the proof of the original specification, or the proof of its negation, at the selected candidate.

The companion implements the parsing, model computation, four fixed frames, certificate reconstruction, and concrete negative evaluation in Python. It emits Lean source candidates for positive cases. It does not implement the Lean correspondence theorem or connect its model to Leant's Exference term graph. That missing correspondence is an explicit integration gate, not a minor matter of converting a JSON response.

The current Length adapter already distinguishes source association from model-relative behavior and treats product-domain queries separately [7]. Heather's realizable-frame package can sit after that boundary, provided its model is proved to describe the same source terms and provider environment. It should not bypass the existing handoff's refusal conditions.

9.3 Provider laws must be attached to their providers

Suppose a provider h is abstracted by a length law $|h(x)| = a|x| + b$. A passive record containing (a, b) is a hypothesis about the model, not evidence that this exact h satisfies the law. Heather needs a Lean theorem

$$\forall x, \quad |h(x)| = a|x| + b$$

under the admitted domain and structures, together with the mapping from the provider identifier to that h.

There are two honest alternatives when this proof is unavailable. Search may use the law heuristically but may not promote its conclusion to a theorem; or the published result may remain conditional on the provider law. The latter is legitimate mathematics when the assumption is visible. It is not an unconditional verification of the provider.

9.4 Negative outcomes and unfinished sketches

A negative result can be an unsupported model, an exhausted search, a model counterexample, a realized source counterexample, or a proof of the negation of the original specification. Only the last two, connected to the exact candidate, justify source-level refutation. Even a perfectly reconstructed abstract solver assignment remains model-relative when a provider law has not been established.

For a sketch with possible completions C, one failed completion c_0 proves only $\neg\mathsf{Spec}(c_0)$. Rejecting the entire sketch requires a proof such as

$$\forall c \in C, \quad \neg\mathsf{Spec}(c),$$

not merely a counterexample to one candidate chosen by the synthesizer. Backward requirement propagation may rule out a class of completions, but the coverage of that class is itself part of the proof.

9.5 Replay the actual edit

The final editor action must replay the exact selected source replacement in the destination context and verify all resulting proof obligations. Rechecking a semantically intended candidate while inserting a different rendered variant is insufficient. The retained record identifies the interpreted statement,

selected candidate, exported proof, dependencies, and policy at acceptance. On a toolchain or import change, replay creates a new record rather than relabeling an old one as current.

10 Certified computer algebra as an interface service

10.1 Three acceptable routes and one forbidden shortcut

Heather can use an algorithm with a proved correctness theorem, an untrusted algorithm that supplies a certificate to a proved checker, or a tactic that constructs an ordinary Lean proof. The acceptance boundary in every case is the requested relation on the original objects. A numeric answer, a solver's success flag, or an equality about a different reified expression is not enough.

For a certificate route, the shape is

$$\mathrm{check}(x, y, c) = \mathsf{true} \quad \Longrightarrow \quad R(x, y).$$

The source-to-expression bridge and the proposition that the actual checker accepted the actual certificate are additional obligations. A checker soundness theorem alone says nothing about an untrusted native process's returned bytes. The default route should use kernel reduction where practical or reconstruct an ordinary proof. Accelerated native evaluation needs its own explicit, versioned trust policy and axiom audit [1, 2, 13].

The forbidden shortcut is to place arbitrary computer-algebra simplification inside definitional equality. A false or incomplete result must fail as a proof attempt, not change what the kernel means by type conversion.

10.2 Typed reification rather than one untyped universal AST

A reusable arithmetic reifier should return a syntax tree together with its interpretation bridge in a fixed algebraic structure. The same ring-expression syntax can support several compatible checkers. It should not erase the distinction between natural-number length arithmetic, integer exact division, field inversion, and real inequalities merely because all four print as familiar arithmetic.

For example, a polynomial identity checker can validate a certificate $p = \sum_i c_i q_i$ by exact normalization under a fixed commutative-ring interpretation. Transport through a ring homomorphism then uses its theorem. The reverse implication for ideal membership need not hold after extending scalars. Likewise, casting an integer quotient is not automatically field division; Section 11 isolates the missing interface.

The first implementation should reuse existing Lean arithmetic tools before building a larger checker. The round-three Schist checker is a useful historical seed for a tiny affine equality chain, not evidence that a multivariate polynomial reifier is already available [1, 2]. The practical question is whether an external service supplies capabilities or reusable certificates that equally equipped Lean does not already provide.

10.3 Result strength and quantitative indices

A service request fixes the kind of result. A witness of $P(x)$ is different from a complete characterization of all x satisfying P. A factorization identity is different from a factorization together with irreducibility and normalization guarantees. An upper error bound is different from a sharp constant. Heather

represents these distinctions by predicates and dependent indices, not a single ladder of result confidence.

Finite precision can also be part of an interface. For formal series, define $f \equiv_N g$ to mean equality of coefficients of degrees below N. For $N \geq 1$, formal differentiation gives

$$f \equiv_N g \quad \Longrightarrow \quad f' \equiv_{N-1} g'.$$

Indeed, coefficient j of a derivative is $(j+1)$ times coefficient $j+1$ of the original series. This is a theorem-backed precision change, not an implicit loss of information hidden by the notation. Integration needs a separate interface with the necessary scalar-division assumptions.

Similarly, if f is L-Lipschitz, $d(x, x_0) \leq \varepsilon$, and an approximation y_0 satisfies $d(f(x_0), y_0) \leq \delta$, the triangle inequality gives

$$d(f(x), y_0) \leq L\varepsilon + \delta.$$

The source can request this enclosure; Heather propagates the quantitative requirements and retains the bound proof. It may not render the enclosure as an exact equality. These examples show how richer type interfaces can carry behavioral constraints without new kernel-level logic.

11 Worked mathematics I: exact quotients without vacuity

11.1 The inherited benchmark

The two syntheses recommend exact quotient transport as a small, non-vacuous arithmetic use site. Their checked helper development is historical evidence reported by those documents, not a fresh compilation in this archive [1, 2]. The mathematical problem is the following.

Let $a, b \in \mathbb{Z}$ and $p \in \mathbb{N}$ satisfy

$$p \geq 4, \qquad p \text{ odd}, \qquad a \equiv 3 \pmod 4, \qquad 2 \mid b. \tag{5}$$

Set

$$N_2 = b^p - 1 - a^p, \qquad N_4 = -a^p b^p, \qquad q_2 = N_2/4, \quad q_4 = N_4/16,$$

where the quotients are initially integer division. The goals are exactness of those divisions and the corresponding rational cast identities.

Proposition 11.1 (Operation-specific arithmetic premises)**.** *Under* (5)*,* $4 \mid N_2$ *and* $16 \mid N_4$*. The second claim requires only* $p \geq 4$ *and* $2 \mid b$*. Consequently the integer quotients satisfy* $4q_2 = N_2$ *and* $16q_4 = N_4$ *and become the corresponding quotients in* $\mathbb{Q}$ *after casting.*

Proof. Write $b = 2t$. Since $p \geq 4$, $b^p = 2^p t^p$ is divisible by 16 and hence by 4. Since $a \equiv -1 \pmod 4$ and p is odd, $a^p \equiv -1 \pmod 4$. Thus $N_2 \equiv 0 - 1 - (-1) = 0 \pmod 4$. Also $16 \mid b^p$ implies $16 \mid -a^p b^p = N_4$, independently of the remaining assumptions. Divisibility makes the integer quotients exact. A ring homomorphism from $\mathbb{Z}$ to $\mathbb{Q}$ preserves the two balance equations. Since 4 and 16 are invertible in $\mathbb{Q}$, solving those equations gives the claimed rational quotient identities. □

11.2 The type-level interface

An exact-quotient package should expose a balance theorem rather than a vague "division is safe" flag. One possible result interface is

$$\mathsf{ExactQuot}(n, d) = \{q : \mathbb{Z} \mid dq = n\}.$$

A construction from integer division carries its divisibility proof. Casting the balance equation along $\iota : \mathbb{Z} \to K$ produces $\iota(d)\iota(q) = \iota(n)$. Dividing in K requires a unit witness for $\iota(d)$, or a theorem supplying it from the destination structure.

Nonzeroness in $\mathbb{Z}$, regularity in an arbitrary ring, and invertibility in the destination are different facts. A field of characteristic zero is one sufficient destination package; it is not the weakest possible condition. The actual consumer asks for the unit of the denominator's image.

A proposed Heather trace is:

```
establish q2_exact : 4 * q2 = N2
  via integer exact division
establish q2_cast : cast(q2) = cast(N2) / 4
  via transport of q2_exact to Rational
```

The first method asks for $4 \mid N_2$ and the relevant division interface; the second asks for the fixed cast map and the destination unit. The proof engine may derive those premises from the selected arithmetic helper theorems. It may not replace the integer quotient expression by a rational one before justifying their relationship.

11.3 Satisfiable support and diagnostic mutations

The tuple $(a, b, p) = (3, 2, 5)$ satisfies the assumptions and gives

$$N_2 = -212, \quad q_2 = -53, \qquad N_4 = -7776, \quad q_4 = -486.$$

This witness matters because a benchmark run only inside an inconsistent Fermat counterexample package could discharge every goal by contradiction, without exercising any exact-quotient reasoning [1, 2].

Removing oddness permits $(3, 2, 4)$, where $N_2 = -66$ is not divisible by 4. Removing the evenness condition permits $(3, 3, 5)$, where $N_2 = -1$. Weakening the exponent bound permits $(3, 2, 3)$, where $N_4 = -216$ is not divisible by 16. Removing the residue condition permits $(1, 2, 5)$, where $N_2 = 30$. These are concrete mathematical counterexamples to the corresponding weakened claims, not merely failed tactic invocations.

For a required arithmetic route, Heather should elaborate its helper theorem in a clean, dependency-closed telescope containing the advertised arithmetic assumptions. This prevents accidental use of unrelated local contradictions. A syntactic dependency audit can enforce that support restriction; it does not establish that every allowed premise is mathematically indispensable, nor that a theorem constant appearing decoratively constitutes the advertised method.

The executed Nat use-site demonstration in this archive is deliberately smaller: it inserts a nonzero proof from positivity and retrieves a divisibility assumption. It exports a proof of the two readiness premises. It neither constructs the Frey quotients nor proves the integer-to-rational casts.

12 Worked mathematics II: locality and analysis

12.1 The autonomous-derivative theorem in full

Let $U \subseteq \mathbb{R}$ be open, let $W, \varphi : \mathbb{R} \to \mathbb{R}$, and let $G_n, H_n : \mathbb{R} \to \mathbb{R}$ be families. Assume that for every $x \in U$, W is differentiable at x with derivative $\varphi(W(x))$; for every n and $x \in U$, G_n is differentiable at $W(x)$ with derivative $H_n(W(x))$; and

$$G_0(W(x)) = W(x), \qquad G_{n+1}(W(x)) = H_n(W(x))\varphi(W(x)).$$

These are the mathematical roles present in the ProveIt theorem, with the supplied derivative family renamed H to avoid confusing a name with a derivative operator [3].

Theorem 12.1 (Range-local autonomous differentiation)**.** *For all $n \in \mathbb{N}$ and $x \in U$,*

$$D^n W(x) = G_n(W(x)).$$

Proof. Induct on n, retaining the universal quantifier over $x \in U$. For $n = 0$, $D^0 W(x) = W(x) = G_0(W(x))$ by the initial equation.

Assume the assertion for n. Fix $x \in U$. Since U is open, it contains a neighborhood of x. On that neighborhood the induction hypothesis identifies $D^n W$ with $G_n \circ W$. The latter is differentiable at x by the chain rule, with derivative

$$H_n(W(x))\varphi(W(x)) = G_{n+1}(W(x)).$$

Equality on a neighborhood transfers this derivative to $D^n W$. Its derivative at x is $D^{n+1}W(x)$, proving the successor assertion. □

The proof does not require G_n to be differentiable away from $W(U)$. It does not differentiate a merely pointwise equality at one point. In a Lean translation, differentiability evidence must be retained rather than inferring it from equality of totalized derivative values alone.

12.2 The consumer interface and its failure neighbor

The critical method is a local derivative transfer. Its inputs are two fixed functions, a point, a neighborhood-equality proof, and an appropriate derivative statement. A convenience adapter obtains neighborhood equality from equality on an open set plus membership. The source file already performs exactly this composition using existing Lean lemmas [3].

Heather's contribution is to infer those adapters under the operation's requirements and explain failures. If openness is removed, it should report that the selected route lacks a neighborhood; another route may still establish one. If only $f(x) = g(x)$ is supplied, differentiation is invalid: $f(t) = t$ and $g(t) = 0$ agree at zero but have different derivatives there. If the induction hypothesis was prematurely specialized to the selected point, Heather should identify that missing uniformity rather than ask the author to experiment with arbitrary tactic variants.

12.3 Almost-everywhere equality has different consumers

The corresponding integration adapter has a different relation. Mathlib's Bochner integral API provides an almost-everywhere congruence theorem for a fixed measure. Its total integral definition and congruence interface should not be confused with separate integrability requirements needed by other operations [14].

Thus an evidence row containing $f = g$ almost everywhere with respect to μ can discharge an integral-congruence requirement for μ, but it cannot justify evaluation at a selected point or an integral with respect to an unrelated measure. Equality off a singleton illustrates the distinction: under Lebesgue measure a point can be negligible, whereas under a Dirac measure at that point it is not. The measure is part of the proposition's identity.

Likewise, exchanging a limit with an integral requires the premises of an applicable interchange theorem. Domination is one sufficient route, not a universal necessary condition for all possible interchange arguments. Heather should report the premises of the chosen route without overgeneralizing the reason for a particular refusal.

13 Worked mathematics III: a whole construction carries laws

For a finite family $(H_i)_{i \in I}$ of commutative A-algebras, consider the functor

$$T \longmapsto \prod_{i \in I} \mathrm{Hom}_{A\text{-alg}}(H_i, T).$$

A finite tensor construction represents this functor. In the cited Fermat source, each H_i is additionally finite and free as an A-module, and the constructed representing algebra has those properties [4].

The proof's essential recursive interface is precise. The empty family is represented by A. If W_0 represents a smaller family, then $W_0 \otimes_A H$ represents that family with one additional factor. Given two algebra maps into a commutative target, the tensor universal property provides the combined map. The equivalence of homomorphism spaces is natural because postcomposition commutes with restriction to each factor. Finite freeness is preserved at the tensor step: with finite bases of sizes m and n, the pure tensors of basis vectors give a basis of size mn. This supplies a mathematical route for the relevant closure property, while the source uses Lean's module interfaces to assemble it [4].

A useful Heather package would expose the representing equivalence, naturality, and module properties as a dependent interface for the one constructed W. It would not independently synthesize one W for finiteness, another for representability, and then merge their rows by printed name. Its naturality facet would quantify over target algebras, their structures, postcomposition maps, and the actual representing equivalence.

The missing-factor hypothesis has a simple negative neighbor. For a singleton family over $A = \mathbb{Q}$ with $H = \mathbb{Q}[X]$, the functor is already represented by H. A natural representing isomorphism forces any other representing algebra to be isomorphic to H: apply the natural isomorphism and its inverse to identity maps to obtain mutually inverse algebra maps. But the powers $1, X, X^2, \ldots$ are linearly independent, so H is not a finite-dimensional $\mathbb{Q}$-module. Finiteness of the index set alone therefore cannot justify the finite-module facet.

This example also separates existential mathematical construction from executable data extraction. The source theorem is an existence statement. A proof may introduce its existential witness within a proposition-valued argument. An API promising an executable enumeration, explicit bases, or a

computable normalization routine needs additional data, or an explicitly noncomputable construction policy. A proposition asserting finiteness is not itself a program that enumerates the carrier.

14 Relation to existing type-system ideas

Refinement types and type-directed proof obligations are established ideas, not Heather inventions. Liquid Types combine type inference with predicate abstraction to infer useful refinements automatically. Heather shares the interest in finite, disciplined inference, but it does not claim to make arbitrary Lean propositions decidable or to infer a principal refinement for a mathematical development. Its automation is deliberately incomplete outside a selected, theorem-backed fragment [15].

Occurrence typing makes the classification of an expression depend on the facts available at its occurrence, particularly in control flow. Heather's branch-local facets have a related motivation: a hypothesis learned in one branch changes what can be established there, not what may be assumed in its sibling. Heather uses retained Lean evidence and explicit dependent contexts rather than treating this analogy as a ready-made implementation [16].

Within Lean, automatic parameters, property tactics, dependent structures, and program-verification tools already cover important pieces of the proposed interface. Heather's research question is whether a uniform, mathematically legible protocol for those pieces improves proof construction and repair. The strongest alternative is not bare tactic syntax with no libraries; it is well-engineered ordinary Lean with the same semantic packages and services [9, 10, 11].

The two round-three reports make this distinction central. Their consensus calculus is a foundation to reuse; their implementation questions are where Heather seeks to add specificity. The realizable-frame package, in particular, is a domain-adaptive use of elementary finite-dimensional linear algebra, not a claim that finite testing proves unrestricted programs correct [1, 2].

15 How far should Lean's type system be extended?

15.1 Four different meanings of "extend"

Heather distinguishes four interventions that are often conflated. A library extension adds predicates, bundled structures, and theorems. An elaboration extension changes how surface types and omitted arguments are interpreted. A kernel extension adds new primitive term or type constructors with checking rules. A conversion extension changes which expressions count as definitionally equal.

The first two already suffice to express and implement the proposed mathematical interfaces. Lean's existing dependent types can state the contracts; Heather changes which evidence is routinely carried to a use site and how it is shown. That is a real extension of the author's typing discipline, but not an increase in the logic's expressive power [8, 1, 2].

15.2 A precise primitive refinement alternative

A plausible kernel experiment would add, for $A : \mathsf{Type}_u$ and $P : A \to \mathsf{Prop}$, a primitive $\mathsf{Ref}(A, P)$ with operations

$$\frac{a : A \quad h : P(a)}{\mathsf{pack}(a, h) : \mathsf{Ref}(A, P)},$$

$$r : \mathsf{Ref}(A, P) \Longrightarrow \mathsf{value}(r) : A, \qquad \mathsf{evidence}(r) : P(\mathsf{value}(r)),$$

and reduction rules

$$\mathsf{value}(\mathsf{pack}(a, h)) \equiv a, \qquad \mathsf{evidence}(\mathsf{pack}(a, h)) \equiv h.$$

Subsumption would still require an explicit proof. Given $k : \forall x,\ P(x) \to Q(x)$, define

$$\mathsf{weaken}_k(r) = \mathsf{pack}(\mathsf{value}(r), k(\mathsf{value}(r), \mathsf{evidence}(r))).$$

This keeps the underlying value fixed. It does not insert an oracle for $P \Rightarrow Q$ into type checking.

The evident translation is to Lean's existing subtype $\{a : A \mid P(a)\}$, with its ordinary constructor and projections. On these rules the primitive is therefore a candidate representation optimization, not new mathematics. Introduction still requires h: making the evidence unnecessary would either move the proof obligation elsewhere or change the logic. The implementation experiment would need to establish preservation of typing and reduction under translation, alongside the ordinary metatheoretic and trusted-code obligations of a kernel change.

No measured encoding obstruction in the inspected examples demands this primitive. Local facets avoid repeated packaging already. Before implementing it, profile ordinary subtypes, structures, proof sharing, reducible adapters, and existing elaborator facilities on the same workload. A native constructor must demonstrate a benefit those alternatives cannot provide.

15.3 Why not semantic subtyping in conversion?

A rule that silently accepts $A \mid P$ wherever $A \mid Q$ is expected whenever $\forall x,\ P(x) \to Q(x)$ is true would couple conversion to an unrestricted mathematical entailment problem. A bounded prover can implement a useful *elaboration attempt*, but its timeout cannot decide that the subtype relation is false. A kernel whose conversion depends on such attempts would make a basic checking operation sensitive to search policy.

Equality reflection creates a related problem: a propositional equality would be promoted into conversion rather than used by an explicit transport term. Heather needs proof-directed transport, not that promotion. Keeping the proof step explicit in the core preserves a small acceptance boundary while allowing its routine surface syntax to be omitted.

Restricted new definitional laws, such as laws intended to reduce transport or functoriality overhead, remain a research option. They must be justified by a specific cost and a metatheory for the exact rule set. Heather does not infer that they are harmless merely because their propositional counterparts are provable.

15.4 Behavioral and graded types without a new conversion theory

A function can be exported as a subtype of functions satisfying a contract, a structure with a function field and laws, or an unchanged function with separate evidence parameters. The first two are useful at module boundaries; the last is preferable for accumulating local knowledge. These are different interfaces for the same logical content, not literally the same Lean term encoding.

Quantitative guarantees likewise fit dependent indices: a finite jet's order, an enclosure's radius, a Lipschitz constant, or a permitted domain. Operations have theorems transforming these indices. Their transformations need not all form a single semiring or scalar strength order; heterogeneous mathematical relations deserve heterogeneous indices.

For resource-sensitive programs, state-transition specifications or an appropriate linear interface may be useful. That does not make mathematical proofs themselves single-use resources. A host receipt's epoch can regulate association with one request, while a theorem remains duplicable evidence. Heather keeps these two notions of constraint separate.

16 The delivered executable experiment

16.1 What exists

The archive contains a standard-library-only Python implementation and five small source examples. Its implemented language is intentionally smaller than the proposed narrative surface:

```
heather diagonal_balance
domain diagonal
inputs xs, ys
claim length(append(xs, ys)) == length(append(ys, ys))
```

It parses the source into a bounded expression tree, checks the available list constructors and inputs, computes affine length models, selects a fixed domain-specific frame, constructs a result certificate, and rechecks that certificate against the original request. Positive results emit ordinary Lean proof candidates; negative results include actual lists whose evaluations have different lengths.

A separate small context model resolves the two premises of an exact-quotient consumer over $\mathbb{N}$. It uses only positivity/nonzeroness implications and explicit divisibility assumptions. This is enough to exercise proof-argument insertion, exact object matching, absent-premise behavior, and snapshot refusal. It is not a general rule registry or exact-division implementation.

16.2 What ran

The delivered run used Python 3.13.5 and passed all 18 test methods. The log and machine-readable results are included. Counts are separated by what they measure:

Executed check category	Count
Test methods completed with no failures or errors	18
Concrete expression-length versus affine-model comparisons	7,500
Admissible-domain evaluations in generated law checks	7,200
Concrete negative witnesses checked in generated examples	466
Deliberately corrupted result certificates refused	14
New Lean compilations performed	0

The generated cases use fixed random seeds and small finite length ranges. They are not 14,700 proved theorems. The universal mathematical argument for the intended algorithm is Lemma 8.3 with Theorem 8.2; the correspondence between the Python program and that argument has not been mechanically verified.

The five named source examples produce four positive model conclusions and one concrete counterexample. Constant-empty output preserves length over `List Empty`, but not over `List Nat`. The diagonal contract is accepted under equal input lengths; removing the guard produces a concrete counterexample. Reversal/append and even-domain reversal examples exercise ordinary positive cases.

Constructing a singleton of `Empty` is refused as a typing error, not mislabeled as a mathematical counterexample.

The use-site demonstration resolves two requirements with one implication-rule firing and three retained evidence entries, counting the two assumptions. These are trace statistics for a toy example. They are not measurements of a native Lean evidence index, proof-term growth on a real file, or author productivity.

16.3 Trust and implementation boundaries

The certificate rechecker reconstructs model and frame information instead of trusting serialized coefficients or a result flag. It also evaluates the actual expression on a negative witness. However, producer and checker share parser, expression, and model code. They are not independent implementations, and neither is a verified Lean checker.

The generated Lean files are labeled uncompiled. A working Lean toolchain was not available in this environment, and an attempted toolchain retrieval did not succeed. The archive therefore makes no new kernel-checking claim. It contains reproducible commands for attempting compilation in an appropriate Mathlib project rather than a fabricated successful build log.

General expression rewriting, checked context migration, automatic frame discovery, arbitrary provider contracts, a full `requires` parser, and the Leant handoff remain design specifications. The Python context uses exact anchors and refuses transport. That narrower behavior is a deliberate boundary of the experiment, not evidence that the more general operations have been implemented.

16.4 Why this still tests a useful design question

The experiment establishes an executable counterpoint to coefficient equality on an unrestricted observation space. The same source grammar gives different correct outcomes when its domain changes; impossible abstract witnesses do not become source refutations; and corrupted or mismatched certificates are not accepted merely because a producer labeled them successful.

These results justify attempting a native semantic bridge for the small fragment. They do not yet justify a new proof language. The next gate is a Lean-checked interpretation theorem and exact-candidate correspondence, followed by a genuine comparison against the strongest ordinary-Lean route.

17 Adversarial acceptance and the evaluation contract

17.1 Positive neighbors prevent a trivial refusal system

Every negative test needs a nearby admitted case and an expected failure kind. Otherwise an implementation that rejects everything can appear safe. The following table separates what the companion executes from important native tests still needed.

Mutation	Correct response and positive neighbor	Status
Remove divisibility from a use site	Leave that premise unresolved; resolve it when the matching assumption is supplied.	Executed toy resolver.
Reuse positivity for another object	Refuse cross-anchor reuse; accept the same object in the same snapshot.	Executed.
Reuse a proof after a rebuild	Refuse old epoch; reconstruct evidence in the new context.	Executed.
Use positive length for `List Empty`	Admit constant-empty length law; refute its `List Nat` version with a singleton.	Executed.
Forget equal-length correlation	Recompute the domain frame; do not reuse the old request's result.	Executed.
Change coefficients, frame, or success label	Refuse certificate; accept the unaltered certificate for the original request.	Executed.
Differentiate point equality	Reject that route; accept neighborhood equality with the derivative evidence.	Written counterexample; native test pending.
Change measure beneath an a.e. fact	Require a new relation or checked transport; accept the same measure's integral congruence.	Native test pending.
Drop a factor's module finiteness	Do not infer a finite representing algebra; accept the finite-free family.	Written counterexample; native test pending.
Cast an inexact integer quotient	Require exactness and destination units; accept the non-vacuous exact-quotient fixture.	Written arithmetic; native test pending.

Publication also needs tests for unused false intermediate assertions, changed quantifiers, a wrong branch or precision, decorative use of a required method, unlinked provider laws, and a correct proof of the wrong target. Passing the small companion suite does not establish these document-level properties.

17.2 Separate the library, services, type interface, and prose

The study should compare the following arms, sharing mathematical packages and implementation effort where appropriate:

Arm	Capability allocation
L_0	Ordinary Lean with its existing automation and idiomatic proof style.
L_1	L_0 plus the same newly developed mathematical interfaces, model theorems, and relevant theorem registrations supplied to Heather.
L_2	L_1 plus the same added synthesis, certificate, and requirement-resolution services, callable explicitly from ordinary Lean.
L_3	L_2 plus Heather's automatic use-site proof role, fixed-meaning protocol, shared evidence interface, and obligation explanations.
L_4	L_3 plus the compact narrative input surface.
R	A read-only mathematical renderer over the same checked Lean developments, without a separate input language.

Ordinary Lean automatic parameters and reasonable thin wrappers are allowed in the controls. A comparison that forbids those facilities would manufacture an advantage for Heather. If a small Lean library reproduces the useful interface, the correct conclusion is to ship that library rather than insist that a new language has been validated.

The decisive comparisons are L_3 versus L_2 for the typing interface, L_4 versus L_3 for narrative input, and R versus the input-language arms for reading benefits. Gains from a new mathematical theorem belong to the library arm, not to syntax.

17.3 Tasks, measurements, and cost accounting

The inspected autonomous-derivative, tensor, and exact-quotient examples are development tasks. They cannot become held-out evidence by being renamed. After the interface and registrations are frozen, a maintainer should select new related tasks and mutations without showing them to the implementation team in advance. Both successful and unsuccessful tasks must be reported.

A counterbalanced study should use matched but distinct tasks so that a participant's second attempt does not merely reuse the first proof from memory. Record authoring time, interventions, failed repairs, proof-reading errors, and success under a fixed assistance budget. Machine measurements should separately record elaboration time, rule-generation time, rule firings, proof term size, checking time, cache size, and rebuild cost. Kernel check time must not be conflated with external search time.

Infrastructure costs include wrapper lemmas, registration metadata, provider proofs, frame coverage theorems, reification bridges, training, and maintenance after library changes. An amortized cost claim needs a stated number and kind of future uses. A spectacularly short surface example with a bespoke package behind it is not by itself a compression result.

The reading test should precede access to the expanded view. Readers should recover the quantifier order, carrier and structures, domain restrictions, result strength, and assumptions from the compact statement. An observed zero error count is desirable but does not prove semantic perfection; adversarial checking and formal interpretation work remain necessary.

17.4 Answers to the second synthesis's ten implementation questions

Table 1 uses the T1–T10 questions in *Nine Refinements* as a compact accountability ledger [2]. The other synthesis's concerns about source fidelity, non-vacuous fixtures, exact provider evidence, realization, and equally equipped baselines are addressed by the same design boundaries throughout the article [1].

Table 1: What Heather answers, and what remains to be measured.

Id	Question	Answer in this proposal
T1	Cost of a real evidence index?	Not measured. The delivered trace is a toy; native proof-size, timing, and snapshot data remain an acceptance gate.
T2	Which rules fire, versus lookup?	One positivity implication fires in the toy. A native registry must log use and compare against search and existing property tactics.
T3	Does simplification break anchors?	Only definitionally equal reuse, checked equality/iff transport, or a registered relational consumer is allowed. The toy refuses general rewriting.
T4	Can reification be shared?	Share typed arithmetic syntax and interpretation where structures and semantics agree; do not use one untyped expression model for all domains. No general reifier was built here.

Id	Question	Answer in this proposal
T5	Completeness of Length abstractions?	Realizable frames give a written criterion and four executed domain instances for the stated affine grammar, not a theorem about every Leant Length contract.
T6	How does a.e. reasoning enter?	As a relation indexed by its measure, with an integral-congruence consumer and no automatic point-evaluation consumer.
T7	Do lexical structures shorten preambles?	Unmeasured. Heather freezes structures for fidelity; it does not claim to remove all instance setup.
T8	What bounds rule generation?	A finite typed term pool, bounded constructors, finite parameter instantiation, and explicit attempt limits precede closure.
T9	What do compact-view readers recover?	A proposed blind reading protocol; no study was run.
T10	Is one implementation enough?	No independent implementation is delivered. The Python producer/rechecker share code; a native checker and independent adversarial implementation are still needed.

18 Roadmap, stopping rules, and final recommendation

The next implementation should be a Lean package with a narrow public API, not an ambitious parser. First, mechanize the list grammar interpretation and the realizable-frame theorem, with the four admitted domains and exact source correspondence. Export a theorem or a concrete refutation for the original candidate. This is the behavioral slice for which the present article supplies both a mathematical argument and executable development examples.

Second, add one property-implicit consumer over the already developed exact-quotient interface. Resolve its designated proof parameters, retain its fixed structures and source target, and measure it against equally equipped Lean automatic parameters and explicit service calls. Include the satisfiable fixture and the arithmetic mutations. A failed premise should remain a typed obligation with its consuming operation.

Third, implement a small set of checked equality, proposition-equivalence, and neighborhood adapters. Use the autonomous-derivative theorem as a development test for preserving a quantified induction motive and range-local hypotheses. Rebuild the index per declaration before attempting persistent context migration. Then move to a genuinely new analysis task selected after the implementation freezes.

Only after these gates should Heather add the narrative syntax illustrated in Section 3. A renderer can be developed earlier because it can expose the same typed interface without creating a new input language. A kernel primitive comes later still, and only if profiling identifies an encoding cost the existing facilities cannot adequately remove.

The stopping rule is substantive. If L_1 explains the gains, retain the mathematical library. If L_2 explains them, retain the services. If R gives the reading benefits, retain the renderer. If L_3 improves use-site authoring but narrative input does not, ship the Lean extension without a separate language. Each outcome preserves useful work; none requires claiming an unobserved victory for Heather.

18.1 Conclusion

Heather treats concise mathematics as a problem of well-designed interfaces, not of trusting omitted reasoning. Its objects carry proved local facets; its operations demand precise evidence; its behavioral contracts refer to actual functions and actual admissible inputs; its proof search cannot alter the requested meaning. The ordinary Lean kernel remains the acceptance boundary.

The main mathematical addition is a realizable-frame criterion that makes finite affine reasoning sensitive to domains and correlations. The main operational additions are concrete policies for proof-implicit arguments, finite rule generation, rewriting, snapshot lifetime, and distinct failure statuses. The delivered reference implementation exercises a small portion of those policies and explicitly stops short of claiming a native elaborator or Lean verification.

The question for the next stage is consequently testable: does making these interfaces implicit and visible at the right moments reduce mathematical proof-writing and repair costs beyond what the same theorems and services already accomplish in Lean? Heather is worth pursuing to answer that question, not because another name or a larger grammar settles it in advance.

A Source ledger and version boundaries

The following full commit identifiers were returned by the repository interfaces during this review. They identify the source observations used here, not an assertion that the branches will remain at these revisions.

```
Brainstorm
  bf33339059950608 70f8585e297ffc16f3fe7e86
ProveIt
  24ce8bd743eaab64a91ce90725ea00f498d319d2
anthropics/fermats-last-theorem
  aa2d8b34692b16c70f699536de0d8e75b9a3e9ef
Leant
  3a40904be8a410d832d9ab6900b3d3e7b425eccb
```

The two requested main report sources are `docs/round-3/synthesis/unified-report.tex` and `docs/round-3/unified_report/unified_report.tex` in Brainstorm. Their reconciled discussions, mathematical interfaces, source critiques, and implementation questions are the basis of the inheritance analysis in this article. This is not a claim of independent review of every proposal or every separate appendix artifact referenced by those sources.

The focused Lean source study reads `Analysis/FabiusFunction/Lean/FabiusFunction/AutonomousIteratedDeriv.lean` in ProveIt and `P2M/Sol/S_Algebra_exists_algHom_equiv_pi.lean` in the Fermat repository. The focused Leant study reads `src/Leant/Synth/Verification.hs`, `src/Leant/Synth/PostVerification.hs`, and `src/Leant/Synth/Length/Adapter.hs`.

ProveIt's `lean-toolchain` at the observed pin specifies Lean 4.32.0. The historical compilation claims in the syntheses concern their documented runs and retained experiments. The current online Lean reference consulted here identifies a different documentation version, 4.34.0-rc2. Heather's uncompiled exports do not inherit a successful build result from either source. The distinct Leant snapshot `823259f7` discussed by the reports is not silently substituted for the `main` revision observed here.

B Minimal native interface contract

A native implementation should start with a small semantic API. The following is interface pseudocode, not compiled Lean syntax:

```
EvidenceEntry:
  source_context : dependent telescope
  proposition    : elaborated Prop
  proof          : term checked at that proposition
  origin         : source assertion or registered application

Requirement:
  context        : current dependent telescope
  target         : fixed elaborated Prop
  consumer       : operation and designated parameter
  policy         : rules, providers, budgets, permitted axioms

Outcome:
  proved(proof_artifact)
  unresolved(frontier, exhaustion_information)
  refused(schema_or_context_mismatch)
  refuted(proof_of_negation)
```

A model counterexample belongs to a provider-specific intermediate result, not to `refuted` until the source bridge and admissibility proof are present. Request routing information and local snapshot identifiers accompany these objects in the implementation, but do not substitute for any field above.

For the affine package, a mathematical native interface can use a base input $s_0 : S$, a finite family $s_j : S$, and a proof that every observation difference is a rational linear combination of their differences. It also needs the expression interpretation theorem and the correspondence to the actual candidate. The simplest implementation should accept only explicitly registered domain packages with those proofs. Automatic discovery of the admissible image is not a prerequisite for the useful four instances in Section 8.

The native resolver should have a plain Lean invocation path in addition to Heather's automatic proof-parameter path. This is both a practical escape hatch and the strongest baseline for separating inference capability from use-site convenience.

C Archive contents and reproduction

`Heather.tex` is self-contained LaTeX source. It does not require downloaded fonts, external figures, or a separate bibliography database. `Heather.pdf` is its compiled rendering. The root README gives the build commands.

The `companion/` directory contains the Python frontend, tests, use-site demonstration, source examples, emitted result certificates, uncompiled Lean candidates, and test logs. The status fields distinguish positive reference-model conclusions from Lean acceptance. No `sorry`-based proof is presented as verification; no successful Lean execution log is included.

To rerun the experiment from that directory:

```
python test_heather.py
python use_site_demo.py
python heather.py examples/diagonal.hthr --output generated
python heather.py examples/empty.hthr --output generated
python heather.py examples/free.hthr --output generated
python heather.py examples/even.hthr --output generated
python heather.py examples/refuted.hthr --output generated
```

The test counts are deterministic; wall-clock timings in the JSON are not. To attempt the exported Lean proofs separately, use a properly prepared Mathlib project and invoke `lake env lean` on each generated file. Such an attempt must be recorded as a new validation result. The package does not claim that compilation in a new environment will be automatic or independent of its pin.

References

[1] *From Properties to Proof Obligations: Round 3 Synthesis*. Brainstorm campaign report, 6 September 2026. `docs/round-3/synthesis/unified-report.tex`. Pinned source. The source's historical experiments are attributed to the report, not rerun here.

[2] *Nine Refinements: Property-Aware Typing in a Mathematician's Proof Language over Lean*. Brainstorm campaign report, 6 September 2026. `docs/round-3/unified_report/unified_report.tex`. Pinned source.

[3] Vladimir Reshetnikov and ProveIt contributors. *Iterated derivatives of a solution of an autonomous equation*. `AutonomousIteratedDeriv.lean`. Pinned source; see `AutonomousODE.iteratedDeriv_eq_comp` and its differentiable-family corollary.

[4] Contributors to *fermats-last-theorem*. `P2M/Sol/S_Algebra_exists_algHom_equiv_pi.lean`. Pinned source; see the two-factor tensor lemma and `solution`.

[5] Vladimir Reshetnikov and Leant contributors. `src/Leant/Synth/Verification.hs`. Pinned source; see `Verified` and the candidate-group verification functions.

[6] Vladimir Reshetnikov and Leant contributors. `src/Leant/Synth/PostVerification.hs`. Pinned source; see the epoch-scoped candidate handles and exact-permutation seal.

[7] Vladimir Reshetnikov and Leant contributors. `src/Leant/Synth/Length/Adapter.hs`. Pinned source; see checked scalar and product-domain query preparation and its stated limits.

[8] Lean contributors. *The Lean Language Reference*. Online reference, consulted 6 September 2026. The consulted landing page identifies version 4.34.0-rc2.

[9] Lean contributors. *The Lean Language Reference: Function Application*. Official documentation, consulted 6 September 2026. Includes automatic-parameter application behavior.

[10] Mathlib contributors. *Mathlib.Tactic.FunProp*. Generated primary API documentation, consulted 6 September 2026.

[11] Lean contributors. *The mvcgen Tactic: Overview*. Official documentation, consulted 6 September 2026.

[12] Lean contributors. *Tactic Reference* and *The grind Tactic*. Tactic reference; grind documentation. Consulted 6 September 2026.

[13] Lean contributors. *The Lean Language Reference: Axioms*. Official documentation, consulted 6 September 2026. Includes the distinction between proof axioms, noncomputable data, parametricity, and trust-sensitive evaluation.

[14] Mathlib contributors. *Mathlib.MeasureTheory.Integral.Bochner.Basic*. Generated primary API documentation, consulted 6 September 2026; see `integral_congr_ae` and the integral definition.

[15] Patrick Rondon, Ming Kawaguchi, and Ranjit Jhala. *Liquid Types*. PLDI, 2008. Authors' publication page and abstract.

[16] Sam Tobin-Hochstadt and Matthias Felleisen. *The Design and Implementation of Typed Scheme: From Scripts to Programs*. ArXiv preprint 1106.2575, 2011. Authors' preprint abstract.